# Dhaka International University

## Department of CSE




# Final Lab Project Report

## KharochKhata: Expense Tracker App

**Course Name:** Mobile Application and Development Lab

**Course Code:** 0613-410

### Submitted By:

**Name:** S.M. Ferdous Oahid Rifat

**Batch:** 85

**Roll:** 58

### Submitted To:

Ekramul Haque Amin

Lecturer, Department of CSE

Dhaka International University

**Date:** 05-11-2026

# 1. Introduction

## 1.1 The Problem

Many students and young professionals spend small amounts of money many times a day on food, transport, mobile recharge and shopping. At the end of the month they cannot say where the money went. Notes on paper get lost, and a spreadsheet is slow to open on a phone, so people forget to write expenses down and the habit stops after a few days.

## 1.2 Goal of the App

**KharochKhata** ("kharoch" means expense and "khata" means notebook) is an Android app that makes recording expenses quick enough to do right after paying. A user creates a personal account, adds each expense in a few taps, sees the total spent, looks back at every record, and can edit, delete or share it. The data is stored in Firebase, so it is safe if the phone is changed and it is private to each user.

## 1.3 Objectives

- Let a user register, log in and stay logged in, with each user's data kept separate.
- Record an expense with title, amount, category, date, note and an optional receipt photo.
- Show all expenses in a scrolling list and the total spent, updating in real time.
- Allow editing, deleting (with confirmation) and sharing of any expense.
- Use the phone's own features: camera, gallery, accelerometer (shake to add) and other apps.
- Prevent crashes by validating every input, and follow Material Design.

## 1.4 Technologies Used

| Item | Details |
|---|---|
| Platform | Android (minSdk 26 = Android 8.0, targetSdk 34, compileSdk 35) |
| Language and UI | Java 17 and XML layouts, Material Design 3 theme |
| Backend | Firebase Authentication and Firebase Realtime Database (Spark free plan) |
| Build tools | Android Studio, Gradle 8.9, Android Gradle Plugin 8.7.3, Firebase BoM 33.7.0 |
| Version control | Git and GitHub (public repository, Release v1.0 with the APK) |

# 2. Features with Screenshots

## 2.1 The 12 Required Features

The table shows how each required feature is used for a real purpose in KharochKhata.

| # | Required feature | How KharochKhata uses it |
|---|---|---|
| 1 | At least 5 screens with navigation | Six screens: Splash, Login, Register, Home, Add/Edit, Detail. Intents with `putExtra()` |
| 2 | Layouts and basic views | ConstraintLayout, LinearLayout, TextView, ImageView, Button, CardView |
| 3 | Input with validation | `setError()` on Login, Register and Add/Edit; no crash on empty or wrong input |
| 4 | Model class | `Expense.java` with getters and setters |
| 5 | Scrolling list | Home: RecyclerView + `ExpenseAdapter` + ViewHolder |
| 6 | CRUD with dialog and FAB | FAB adds, tap opens details, Edit button edits, AlertDialog confirms delete |
| 7 | Firebase login | Register, login, logout and stay logged in |
| 8 | Firebase database, real-time list | Realtime Database with `ValueEventListener` |
| 9 | Setting saved on the phone | "Remember me" saved with SharedPreferences |
| 10 | Take or pick a photo | Receipt photo from camera or gallery, shown in an ImageView |
| 11 | One sensor | Accelerometer: shake the phone on Home to open Add expense |
| 12 | Open another app | Share an expense (`ACTION_SEND`), Help web link (`ACTION_VIEW`) |

Three features cannot be shown well in a picture, so they are described here: **Remember me** saves the e-mail on the phone and fills it in at the next login; **real-time update** means an expense added on one device appears on another without refreshing; **shake to add** opens the Add expense screen when the phone is shaken on the Home screen (more than 2.7 g, at most once per second).

## 2.2 Screens

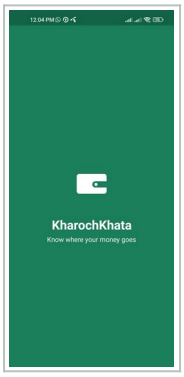



**Figure 1: Splash**

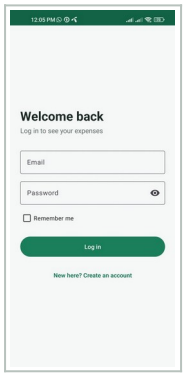



**Figure 2: Login**

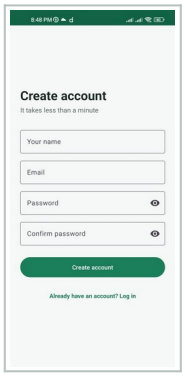



**Figure 3: Register**

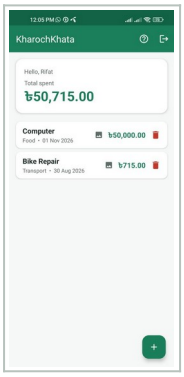



**Figure 4: Home list**

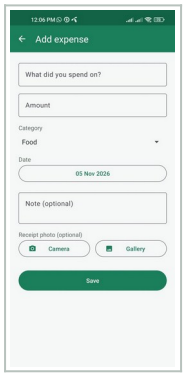



**Figure 5: Add expense**

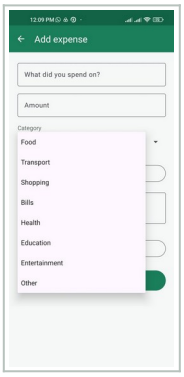



**Figure 6: Category list**

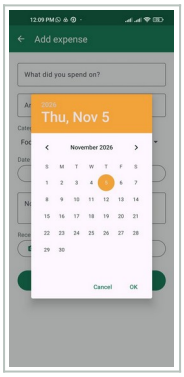



**Figure 7: Date picker**




**Figure 8: Detail**

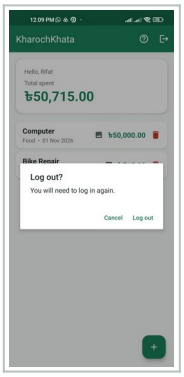



**Figure 9: Log out dialog**

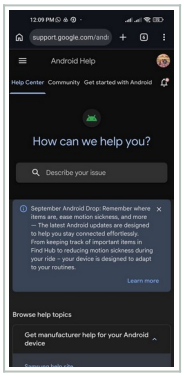



**Figure 10: Help (browser)**




**Figure 11: Share chooser**

Figures 5 to 7: the Add expense form with its category Spinner and DatePickerDialog. Figure 9: the AlertDialog that asks before logging out (delete uses the same dialog). Figure 10: the Help page opened in the browser with `ACTION_VIEW`. Figure 11: the `ACTION_SEND` chooser used to share an expense.

# 3. Components Used, Alternatives and Why I Chose Them

This section lists every component that is really used in KharochKhata. For each one I explain what it does in the app, name its alternatives, and give the reason why I chose it.

## 3.1 Screens and Navigation

### Activities + Intents

**What it does in KharochKhata:** The app has six Activities: Splash, Login, Register, Home, Add/Edit and Detail. Explicit Intents move between them. Home sends the expense id to the Detail screen and to the Add/Edit screen with `putExtra()`. Add/Edit reads it with `getStringExtra()` and switches between Add mode and Edit mode.

**Alternatives:** Fragments + Navigation Component (a single-Activity app).

**Why I chose it:** Each screen is one Java class with one XML layout, so it is easy to read, test and explain. The app is small, so the extra setup of the Navigation Component was not needed.

### Handler (`postDelayed`)

**What it does in KharochKhata:** Keeps the Splash screen visible for 1.5 seconds, then opens Home if `FirebaseAuth.getCurrentUser()` is not null, otherwise Login. The callback is removed in `onDestroy()`.

**Alternatives:** Android 12 SplashScreen API, `CountDownTimer`, `Thread.sleep()`.

**Why I chose it:** A delayed Handler is the shortest way to show a timed Splash screen without blocking the UI thread. Removing the callback avoids running code on a screen that is already closed.

## 3.2 Layouts and Views

### ConstraintLayout

**What it does in KharochKhata:** Lays out the Splash screen: the logo, the app name and the tagline are chained vertically and centred.

**Alternatives:** LinearLayout, RelativeLayout, FrameLayout.

**Why I chose it:** Constraints place the three views relative to each other and to the screen, so no nested layouts are needed and the screen looks the same on different sizes.

### LinearLayout + ScrollView

**What it does in KharochKhata:** Builds the Login, Register, Add/Edit and Detail forms as vertical stacks, the list rows, and the button rows (equal-width buttons with `layout_weight`). ScrollView wraps the long forms.

**Alternatives:** ConstraintLayout, RelativeLayout, TableLayout.

**Why I chose it:** A form is a simple top-to-bottom list, which LinearLayout expresses clearly. ScrollView keeps every field reachable when the keyboard is open on a small phone.

### CoordinatorLayout

**What it does in KharochKhata:** Root layout of the Home screen. It lets the Snackbar slide up above the floating action button instead of covering it.

**Alternatives:** FrameLayout, RelativeLayout.

**Why I chose it:** It is the layout that Material components such as FAB and Snackbar are designed to work with, and it needs no extra code.

### Basic views (TextView, ImageView, Button, CheckBox, ImageButton)

**What it does in KharochKhata:** TextView shows titles, amounts and notes. ImageView shows the logo and the receipt photo. Button and MaterialButton run save, camera, gallery, edit, share and delete. CheckBox is the Remember me option. ImageButton is the trash icon on each list row.

**Alternatives:** Material-only variants (Chip, Switch, MaterialCheckBox) or custom views.

**Why I chose it:** These standard views cover every need of the app and are well known, so the layouts stay simple and easy to maintain.

### CardView

**What it does in KharochKhata:** Draws the summary card on Home, each expense row in the list and the card on the Detail screen, with rounded corners and a shadow.

**Alternatives:** MaterialCardView, LinearLayout with a shape background.

**Why I chose it:** It gives a clean card look with two attributes (`cardCornerRadius` and `cardElevation`) and groups related information visually.

### MaterialToolbar + options menu

**What it does in KharochKhata:** The top bar on every screen after login. On Home it has Help and Log out icons defined in `menu_main.xml`. On Add/Edit and Detail it has a back arrow.

**Alternatives:** The default ActionBar, a navigation drawer, bottom navigation.

**Why I chose it:** With a NoActionBar theme and an explicit toolbar I control the colours and icons fully. A drawer or bottom bar is too much for an app with one main screen.

### Material Design 3 theme and resources

**What it does in KharochKhata:** `Theme.Material3.Light.NoActionBar` styles the buttons, text fields and dialogs. Colours, all texts (including the category list) are in `colors.xml` and `strings.xml`. Icons are vector drawables and the launcher icon is an adaptive vector icon.

**Alternatives:** AppCompat theme, hard-coded text and colours in the layouts, PNG icons.

**Why I chose it:** One theme keeps all screens consistent. Texts in `strings.xml` are easy to change or translate, and vector icons stay sharp on every screen density.

## 3.3 Input and Data Display

### TextInputLayout + TextInputEditText with `setError()`

**What it does in KharochKhata:** All forms use outlined text fields with a floating hint and a show/hide password icon. Before saving, the code checks the input and calls `setError()`: required fields, e-mail pattern, password of at least 6 characters, matching confirm password, and an amount above 0 and below 1,000,000,000. A `NumberFormatException` is caught, so wrong input never crashes the app.

**Alternatives:** Plain EditText with Toast messages, a third-party validation library.

**Why I chose it:** The error appears under the exact field that is wrong, so the user knows what to fix, and no extra library is needed.

### Spinner + ArrayAdapter

**What it does in KharochKhata:** Lets the user choose one of eight categories: Food, Transport, Shopping, Bills, Health, Education, Entertainment, Other. The list comes from a `string-array` in `strings.xml` through `ArrayAdapter.createFromResource()`.

**Alternatives:** Radio buttons, chips, AutoCompleteTextView.

**Why I chose it:** A drop-down shows eight choices without using much space on the form, and the list can be edited in one XML file.

### DatePickerDialog

**What it does in KharochKhata:** Lets the user pick the date of the expense. The chosen date is shown on a button and saved as milliseconds in the database.

**Alternatives:** MaterialDatePicker, typing the date in a text field.

**Why I chose it:** It is built in, needs no extra library and removes wrong date formats.

### RecyclerView + Adapter + ViewHolder

**What it does in KharochKhata:** Shows the expenses on Home, newest first. `ExpenseAdapter` binds each `Expense` to the row layout `item_expense.xml`. The ViewHolder keeps the row's views, and an `OnExpenseActionListener` interface sends row taps and delete taps back to the Activity.

**Alternatives:** ListView + ArrayAdapter, GridView, Jetpack Compose LazyColumn.

**Why I chose it:** RecyclerView re-uses row views while scrolling, so the list stays smooth with many expenses, and it is the standard list component for this project.

### Model class (Expense)

**What it does in KharochKhata:** Holds one record: id, title, amount, category, note, photo and date. It has an empty constructor and getters and setters, so Firebase can turn it into JSON and back with `getValue(Expense.class)`.

**Alternatives:** A `Map<String, Object>`, a Kotlin data class, a Room entity.

**Why I chose it:** Typed fields prevent spelling mistakes in keys, and one object is passed to the adapter, so the code is shorter and easier to understand.

### FloatingActionButton

**What it does in KharochKhata:** The round + button on Home opens the Add expense screen.

**Alternatives:** A toolbar action button, an extended FAB, a button at the bottom of the list.

**Why I chose it:** It is the standard Material place for the main action: always visible and easy to reach with the thumb.

### AlertDialog

**What it does in KharochKhata:** Asks for confirmation before deleting an expense and before logging out.

**Alternatives:** A bottom sheet, a separate confirmation Activity, MaterialAlertDialogBuilder.

**Why I chose it:** Delete cannot be undone, so a quick yes/no dialog protects the user from accidental taps without leaving the screen.

### Snackbar and Toast

**What it does in KharochKhata:** Snackbar shows errors and results while the user stays on the screen (login failed, data could not load, expense deleted). Toast shows "Expense saved" or "Expense updated" because the form closes right after saving.

**Alternatives:** Dialogs, a TextView on the screen, notifications.

**Why I chose it:** Both are short and do not block the user. Snackbar suits a screen that stays open, and Toast still shows after the form has closed.

## 3.4 Firebase and Local Storage

### Firebase Authentication

**What it does in KharochKhata:** Handles Register (`createUserWithEmailAndPassword`), Login (`signInWithEmailAndPassword`) and Logout (`signOut`). Splash calls `getCurrentUser()`, so a logged-in user goes straight to Home (stay logged in). The user's name is saved as the display name and shown in the greeting.

**Alternatives:** Google Sign-In, phone OTP, own server login.

**Why I chose it:** No server of my own is needed, Google stores passwords safely and Firebase keeps the session on the phone. E-mail and password is the simplest option for a personal expense app.

### Firebase Realtime Database + `ValueEventListener`

**What it does in KharochKhata:** Stores each user's expenses at `users/{uid}/expenses/{expenseId}`. A `ValueEventListener` is added in `onStart()` and removed in `onStop()`, so the Home list and total update by themselves when data changes, also from another device. `push()` makes the id, `setValue()` creates or updates and `removeValue()` deletes.

**Alternatives:** Cloud Firestore, SQLite, Room.

**Why I chose it:** Data is a simple JSON tree and real-time updates need only a few lines. SQLite and Room keep data on one phone only, so they cannot sync. The free Spark plan is enough for this project.

### SharedPreferences

**What it does in KharochKhata:** The file `kharoch_prefs` stores the Remember me check box (true or false) and the e-mail address. If it is ticked, the e-mail is filled in on the next login.

**Alternatives:** SQLite or Room, Jetpack DataStore, a text file.

**Why I chose it:** One boolean and one string do not need a database. SharedPreferences is the simplest key-value store and needs no setup.

## 3.5 Phone Features

### Camera and Gallery (Activity Result API)

**What it does in KharochKhata:** The Camera button uses `ActivityResultContracts.TakePicturePreview` and the Gallery button uses `GetContent("image/*")`. The photo is shown in an ImageView on the form and on the Detail screen. A try/catch handles phones with no camera app.

**Alternatives:** CameraX, `ACTION_IMAGE_CAPTURE` with FileProvider, an image-picker library.

**Why I chose it:** It opens the phone's own camera and gallery apps, so no CAMERA permission and very little code are needed.

### Bitmap resizing + Base64 (`ImageUtils`)

**What it does in KharochKhata:** Shrinks the chosen photo to at most 480 px, compresses it as JPEG (quality 70) and turns it into Base64 text that is saved inside the expense record. `fromBase64()` turns the text back into a Bitmap for display.

**Alternatives:** Firebase Storage with a download URL, saving a file on the phone and storing its path.

**Why I chose it:** New Firebase projects need the paid Blaze plan to use Storage, but this project runs on the free Spark plan. A small receipt photo is only a few tens of KB, so keeping it in the database works.

### SensorManager + accelerometer

**What it does in KharochKhata:** Shake detection on Home. The three axis values are converted to g-force, and when it is above 2.7 g (at most once per second) the Add expense screen opens. The listener is registered in `onResume()` and unregistered in `onPause()` to save battery.

**Alternatives:** Proximity sensor, light sensor, gyroscope, step counter.

**Why I chose it:** Shaking is a quick and natural way to add something, and an accelerometer is present on almost every phone.

### Implicit Intents (`ACTION_SEND`, `ACTION_VIEW`)

**What it does in KharochKhata:** `ACTION_SEND` with a chooser shares an expense as text with any app such as WhatsApp, Messenger or e-mail. `ACTION_VIEW` on the Help menu opens a web link in the browser. Both are wrapped in try/catch for `ActivityNotFoundException`.

**Alternatives:** An explicit Intent to one fixed app, `ACTION_DIAL` or `ACTION_CALL`, a custom share dialog.

**Why I chose it:** The user picks the app, so nothing is hard-coded or must be installed, and no permission is needed.

## 3.6 Tools

### Java + XML in Android Studio

**What it does in KharochKhata:** All logic is written in Java and all screens are XML layouts.

**Alternatives:** Kotlin, Jetpack Compose, Flutter.

**Why I chose it:** It is required by the course, and XML keeps the design separate from the logic, which helped me learn the Android basics step by step.

### Gradle + Google Services plugin + Firebase BoM

**What it does in KharochKhata:** Gradle builds the app. The file `google-services.json` and the Google Services plugin connect it to my Firebase project, and the Firebase BoM keeps the Firebase library versions matching.

**Alternatives:** The Firebase Assistant in Android Studio, adding each library version by hand.

**Why I chose it:** The BoM avoids version conflicts, and the config-file setup is the official way to connect an Android app to Firebase.

### Git + GitHub (Release v1.0)

**What it does in KharochKhata:** Git records every change. The project is public on GitHub with a README, and the APK is attached to Release v1.0.

**Alternatives:** GitLab, Bitbucket.

**Why I chose it:** It is free, widely used in industry and required by the course. The commit history shows my progress.

# 4. Firebase Database Structure

KharochKhata uses two Firebase services on the free Spark plan: **Authentication** (e-mail and password) and the **Realtime Database**. Every user has a separate branch of the database, named with the user's unique id (`uid`) that Firebase Authentication creates.

## 4.1 Data Tree

```
users
  └── {userId}                     (uid from Firebase Authentication)
        └── expenses
              └── {expenseId}        (key made by push())
                    ├── id          : "-Nxyz..."
                    ├── title       : "Lunch"
                    ├── amount      : 250.0
                    ├── category    : "Food"
                    ├── note        : "With friends"
                    ├── photo       : "/9j/4AAQ..."   (Base64 JPEG, optional)
                    └── dateMillis  : 1790000000000
```

## 4.2 Fields of One Expense

| Field | Type | Meaning |
|---|---|---|
| id | String | Unique key of the record, made by `push()`; also stored inside the record |
| title | String | What the money was spent on (maximum 60 characters) |
| amount | double | Amount spent; must be above 0 |
| category | String | One of the eight categories chosen in the Spinner |
| note | String | Optional note (maximum 200 characters) |
| photo | String | Receipt photo as Base64 text (480 px JPEG), empty if there is no photo |
| dateMillis | long | Date of the expense in milliseconds since 1970, used for sorting |

## 4.3 Security Rules

The rules below are published in the Rules tab. A user can read and write only the branch whose name equals his or her own uid, so nobody can see another person's expenses.

```
{
  "rules": {
    "users": {
      "$uid": {
        ".read": "$uid === auth.uid",
        ".write": "$uid === auth.uid"
      }
    }
  }
}
```

## 4.4 How the App Uses the Database

- `AppUtils.expensesRef()` builds the path `users/{uid}/expenses` for the logged-in user.

- Add: `push()` creates a new key, then `setValue(expense)` saves the record. Edit: `setValue()` on the existing key.
- Delete: `removeValue()` on the key, after the AlertDialog confirmation.
- Read: a `ValueEventListener` on Home (list and total) and on Detail (one record) gives real-time updates.

### 4.5 Firebase Console Screenshots

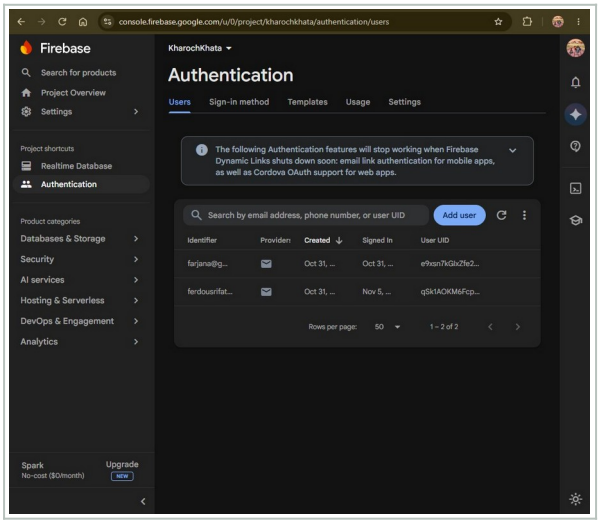



**Figure 12: Authentication users**
Registered users in Firebase Authentication (Security > Authentication > Users).

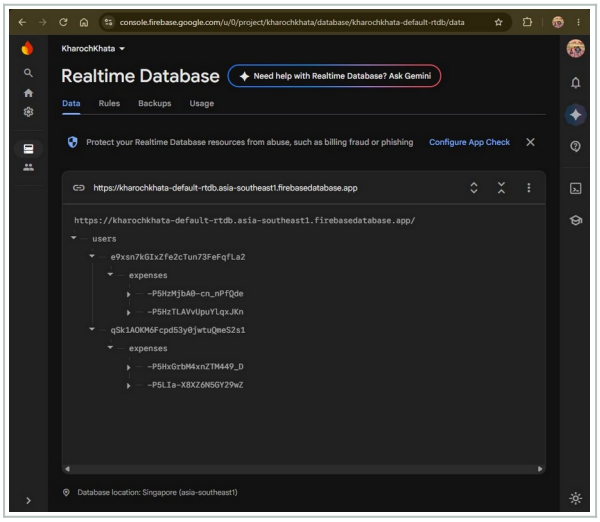



**Figure 13: Realtime Database data**
The `users > uid > expenses` tree in the Realtime Database Data tab.

## 5. GitHub Repository

| | |
|---|---|
| **Repository (public)** | https://github.com/FerdousRifat/KharochKhata |
| **Release v1.0 (APK)** | https://github.com/FerdousRifat/KharochKhata/releases/tag/v1.0 |

The repository is public and has a `README.md` with the app name, my name and ID, the feature list, the Firebase setup steps and the steps to run the app. The project was pushed in six steps (more than the five required), each one adding a working part of the app, so the commit history shows the progress of the work. The table lists the commits exactly as they appear on GitHub, oldest first.

| Push | Commit | Commit message | What was added |
|---|---|---|---|
| 1 | 579fba8 | Initial project setup, login, register and splash activities | Gradle setup, layouts, theme, Splash, Login and Register screens |
| 2 | 14f5185 | Implemented MainActivity and core dashboard structure | Home screen (dashboard) with the main structure |
| 3 | d96589d | Added Expense models, adapter, and add/edit expense logic | Expense model, RecyclerView adapter, add and edit form |
| 4 | 3746596 | Added ExpenseDetailActivity and camera/image selection utilities | Detail screen, camera and gallery photo helpers |

| Push | Commit | Commit message | What was added |
|---|---|---|---|
| 5 | 662a7eb | Finalized complete project integration and added REPORT_OUTLINE.md | Full integration of all features, report outline (force push) |
| 6 | c650508 | Removed target properties and markdown files from remote repository | Clean-up of the remote repository; Release v1.0 is made from this commit |

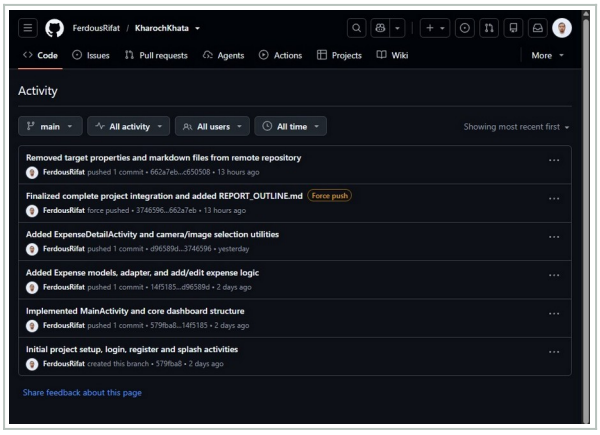



**Figure 14: Commit history**

The commit list (Activity page) of the repository on GitHub.

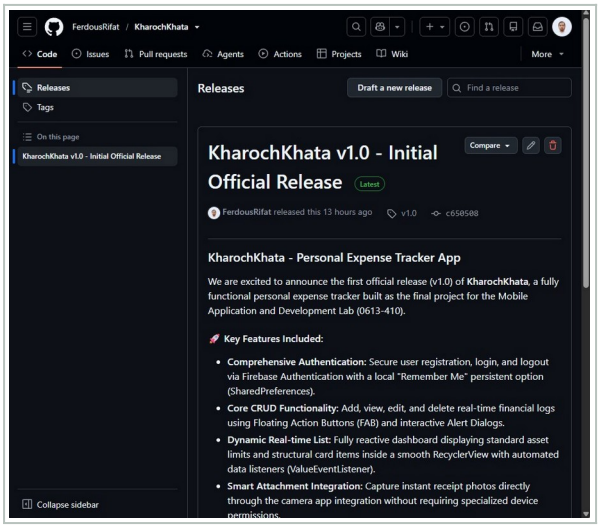



**Figure 15: Release v1.0 page**

Release v1.0 of KharochKhata on GitHub.

# 6. Problems Faced and How I Solved Them

| # | Problem | Cause | Solution |
|---|---|---|---|
| 1 | Build failed: "File google-services.json is missing" (task `processDebugGoogleServices`) | The Google Services plugin needs the Firebase config file inside the `app` folder, and it was not there. | Created a Firebase project, registered the Android app with the package name `com.kharochkhata.app`, downloaded the file and put it in `app/`. |
| 2 | Login failed: "API key not valid. Please pass a valid API key" | A placeholder `google-services.json` that only made the build pass had a fake API key. | Replaced it with the real file downloaded from my own Firebase project, then synced and rebuilt. |
| 3 | Login and register showed "CONFIGURATION_NOT_FOUND" | Authentication was not set up in the Firebase project, or the file came from another project. | In Firebase: Security > Authentication > Get started, enabled the Email/Password sign-in method, re-downloaded the file, then Sync, Clean Project and Run. |
| 4 | Could not find the "Build" menu in the Firebase Console | The new console groups features under Product categories instead of a Build menu. | Found Authentication under Security, Realtime Database under Databases & Storage, and Project settings under Settings. |

| # | Problem | Cause | Solution |
|---|---|---|---|
| 5 | Expenses could not be saved or shown properly | Database rules were still locked, and the config file had been downloaded before the Realtime Database was created, so it had no database URL. | Created the database, published the rules in section 4.3, and downloaded `google-services.json` again. |
| 6 | Git was started in the wrong folder, and `.gitignore` was not found | The project sits in a folder inside another folder of the same name, and the `.gitignore` was in the outer one. | Ran Git in the folder that contains `settings.gradle` and copied the `.gitignore` into it. |
| 7 | `google-services.json` was missing on GitHub | `.gitignore` excluded it, so a clone of the repository could not be built. | Removed that line so the file is committed. The key is not a secret; the data is protected by the database rules. |
| 8 | The course asked for several pushes, but the project was already complete | All work was done in one folder before pushing. | Split the project into working steps and pushed them one after another with clear commit messages. The final history has six pushes. |
| 9 | A large photo can make the database slow or fail | A full camera photo is several MB, and it is saved inside a database record. | Resize to 480 px and compress as JPEG (quality 70) before converting to Base64. |
| 10 | Shake opened the Add screen many times | The accelerometer sends many readings per second during one shake. | Used a threshold of 2.7 g and a gap of one second between two shakes. |
| 11 | Possible crashes on wrong input | Empty fields, text instead of a number, or zero and negative amounts. | Validation with `setError()` on every form and a `NumberFormatException` check on the amount. |

# 7. Conclusion and References

## 7.1 Conclusion

KharochKhata solves a simple everyday problem: it helps a person see where the money goes. The app has six screens, login with Firebase, a real-time expense list, receipt photos, a shake shortcut and sharing, and it uses all 12 required features for a real purpose. Building it taught me how Android screens, data and phone features fit together in one working product.

## 7.2 What I Learned

- How Activities, Intents and `putExtra()` connect the screens of an app.
- How RecyclerView, an Adapter and a ViewHolder show a list efficiently.
- How Firebase Authentication and the Realtime Database work, including security rules and real-time listeners.
- How to use the camera, gallery, a sensor and implicit Intents, and how to handle missing apps without crashing.
- How to use Git and GitHub for regular commits, a README and a release with an APK.
- How to read an error message, find its cause and fix it step by step.

## 7.3 Limitations and Future Work

- The camera button returns a small preview picture, so the gallery gives better quality receipt photos. CameraX or a FileProvider capture could give full-size photos.
- Photos are stored as text in the database. Firebase Storage would be better for a larger app.
- Possible additions: monthly totals and charts, budgets with warnings, search and category filter, offline cache with Room, Google Sign-In, and a currency or weather REST API.

## 7.4 References


1. Course document: Final Lab Project, Instructions for Students, Mobile Application and Development Lab (0613-410), Dhaka International University.
2. Android Developers: Intents and intent filters. https://developer.android.com/guide/components/intents-filters
3. Android Developers: Create dynamic lists with RecyclerView. https://developer.android.com/develop/ui/views/layout/recyclerview
4. Android Developers: Save key-value data (SharedPreferences). https://developer.android.com/training/data-storage/shared-preferences
5. Android Developers: Motion sensors. https://developer.android.com/develop/sensors-and-location/sensors/sensors_motion
6. Firebase Documentation: Authenticate with Firebase using password-based accounts on Android. https://firebase.google.com/docs/auth/android/password-auth
7. Firebase Documentation: Read and write data on Android (Realtime Database). https://firebase.google.com/docs/database/android/read-and-write
8. Material Design 3. https://m3.material.io
9. GitHub Docs: Managing releases in a repository. https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository